

Un projet d'internet of things : qui consiste à récupérer en temps réel les informations concernant la température et le statu des ampoules et évidemment contrôler le statu On/Off de ces ampoules.

*Ces informations sont collectées par deux automates **Wago 750 (Pour la Lumières)** et **Siemens S7-1500 (Température)**, qui lisent la température et le statu des lampes à partir des capteurs, puis les publient ensuite dans des topics dans un Broucker Mosquitto en ligne accessible via : `sr-labor.ddns.net:1883`, avec les identifiants : Email étudiant et le numéro de matricule.*

Le but de ce petit projet est donc écrire une application java avec deux micro services MQTT Micro service et REST Micro service :

- **L'MQTT Micro service** aura pour rôle de se connecter au Broucker Mosquitto et récupérer les données publiées par les deux automates et les enregistre dans une base de données MongoDB

- **Le REST Micro service** à son tour :

- récupérera ces informations stockées dans la base de données puis les envoie à un client front end qui va ensuite les afficher

-Recevra des commandes utilisateur depuis le front end et publiera ces commandes dans le Broucker Mosquitto qui en suite seront lu par les automates et ces derniers, en fonction de la commande, prendront la bonne décision (s'il faut arrêter d'émettre, ou s'il faut allumer ou éteindre une lampe).

Vue d'ensemble du projet :

Il y a deux types de topics :

- Des topics de données (publiés par les SPS(des automates))
- Des topics de contrôle (publiés par le micro service REST, écoutés par les SPS)

Pour la SPS Wago 750 (Lumières)

Wago750/Status

- Qui publie ? → La SPS Wago
- Qui consomme ? → Le micro service MQTT
- Contenu ? → Un Integer sur 2 octets, représenté en BinaryArray
- Rôle ? → Indique l'état actuel des lampes (on/off pour chaque lampe)
 - Ex : 0000000000000011 → lampe 0 et 1 allumées, les autres éteintes
- But ? → Afficher l'état des lampes dans le frontend, ou le stocker dans MongoDB

Wago750/Control

- Qui publie ? → Le micro service REST (via Paho MQTT)

- Qui consomme ? → La SPS Wago
 - Contenu ? → Un Integer parmi 0, 1, 2, 3
 - Rôle ? → Commande de mode
 - 0 : mode repos (la SPS arrête d'envoyer sur Wago750/Status)
 - 1, 2, 3 : changent le mode d'éclairage
 - But ? → Permettre à l'utilisateur, via le frontend, de contrôler la lumière
-

Pour la SPS Siemens S7-1500 (Température)

S7 1500/Temperatur/Ist

- Qui publie ? → La SPS Siemens
- Qui consomme ? → Le micro service MQTT
- Contenu ? → Un Double
- Rôle ? → Température mesurée actuellement
- But ? → Suivi en temps réel + stockage MongoDB + affichage dans frontend

S7 1500/Temperatur/Soll

- Qui publie ? → La SPS Siemens
- Qui consomme ? → Le micro service MQTT
- Contenu ? → Un Double
- Rôle ? → Température cible souhaitée
- But ? → Affichage / comparaison avec température réelle

S7 1500/Temperatur/Differenz

- Qui publie ? → La SPS Siemens
 - Qui consomme ? → Le microservice MQTT
 - Contenu ? → Un Double
 - Rôle ? → Différence entre température cible et température actuelle
 - But ? → Affichage + logique métier possible (alerte si trop d'écart)
-

Microservice MQTT (avec Paho) :

- Il se **connecte au broker Mosquitto** : sr-labor.ddns.net:1883, **authentifié** (mail universitaire + numéro matricule).
- Il s'abonne aux **topics suivants** pour recevoir les données des SPS :
 - Wago750/Status : état des lumières (2 bytes, binaire)
 - S7 1500/Temperatur/Ist : température actuelle
 - S7 1500/Temperatur/Soll : température cible
 - S7 1500/Temperatur/Differenz : écart de température
 - Random/Integer : pour les tests si la SPS est offline
- Il **reçoit** ces messages, les **analyse** (Integer / Double / BinaryArray), et les **stocke dans MongoDB**.
- MongoDB tourne en local et peut être explorée avec MongoDB Compass.

Microservice REST :

- Il expose une **API REST** (via Spring Boot).
- Il propose deux grands rôles :

Lecture :

- Il interroge la base MongoDB pour récupérer les **dernières données des SPS**.
- Il les transmet au **frontend** (ici Angular) pour affichage en direct (températures, état des lumières, etc.).

Commande :

- Il reçoit des **commandes utilisateur** (via HTTP POST) depuis le frontend.
- Il publie ensuite un message MQTT via **Paho** vers le broker Mosquitto :
 - Topic : Wago750/Control
 - Payload : un entier 0, 1, 2 ou 3
 - 0 : mode repos (la SPS cesse d'émettre)
 - 1, 2, 3 : changement du **mode de lumière**

Frontend (Angular) :

- **Il affiche les valeurs actuelles (températures, statut).**
- **Il propose quatre boutons bleus :**
 - "Mode Repos (0)"
 - "Lumière Mode 1"
 - "Lumière Mode 2"
 - "Lumière Mode 3"
- **Lors d'un clic, il fait un HTTP POST vers le microservice REST, qui publie ensuite la commande MQTT.**

Flux de données (côté commande Wago 750) :

1. Utilisateur clique sur un bouton dans le frontend.
2. Frontend envoie une requête POST au microservice REST.
3. Le REST publie le message via Paho MQTT vers le broker Mosquitto (Wago750/Control).
4. La SPS Wago 750 reçoit la commande et change de mode.